

0.1 Churn Prediction System Project Documentation

This document describes the Churn Prediction System project, its structure, how to set it up, and step-by-step instructions to reproduce the data preparation, modeling pipelines, and notebook analyses included in this repository.

0.2 Table of Contents

- Project overview
 - Prerequisites
 - Project structure (key files)
 - Data description
 - How the code is organized (modules & responsibilities)
 - Step-by-step: reproduce the notebook EDA & modeling
 - Step-by-step: run the packaged pipeline (ingestion → transformation → training → prediction)
 - Deployment & serving (Hugging Face Spaces, FastAPI, Docker)
 - Notes, assumptions, and next steps
-

0.3 Project overview

This repository contains an end-to-end churn prediction system for a Telco customer dataset. It includes:

- Exploratory Data Analysis and model experiments in `customer.ipynb`.
- A modular Python package under `src/CustomChurnPrediction` implementing data ingestion, transformation, modeling, utilities, and simple pipelines.
- Artifacts (trained model, preprocessor) written to the `artifacts/` directory.
- A FastAPI-based serving layer for API deployment, containerized with Docker.
- Live deployment on Hugging Face Spaces for interactive predictions.

Primary goal: preprocess Telco customer data, handle imbalance with SMOTE, train and evaluate multiple classifiers (Logistic Regression, Random Forest, SVM, AdaBoost, XGBoost), persist the best model, and serve predictions via a web interface/API.[1][2]

0.4 Prerequisites

- Python 3.9+ (recommend 3.9 or 3.10)
- The repository provides a `requirements.txt`. Install dependencies with:

```
pip install -r requirements.txt
```

Note: If you use a virtual environment, activate it before installing. For Docker deployment, no local Python setup is required build and run the container directly.

0.5 Project Structure (Key Files)

- `customer.ipynb` Notebook with EDA, preprocessing, modeling experiments, and model saving.
 - `src/``Customer_Churn_Prediction/` Main package:
 - `exception.py` Custom exception wrapper for consistent error messages.
 - `logger.py` Logging configuration (writes logs into `logs/` with timestamped filenames).
 - `utils.py` Helper functions to read CSVs, save/load objects, and evaluate models during training.
 - `components/data_ingestion.py` Reads raw CSV (from `notebook/data`), writes `artifacts/raw.csv`, and splits into `artifacts/train.csv` and `artifacts/test.csv`.
 - `components/data_transformation.py` Implements the `TelcoDataCleaner` (custom transformer) and `DataTransformation` pipeline. Handles type conversions, missing values, label encoding, numeric scaling, and SMOTE balancing. Saves the combined preprocessor to `artifacts/preprocessor.pkl`.
 - `components/model_trainer.py` Trains multiple classifiers, runs randomized hyperparameter search, selects the best model, logs metrics (MLflow/Dagshub integration), and saves the final model to `artifacts/model.pkl`.
 - `components/model_monitoring.py` Placeholder file for future monitoring and drift detection logic.
 - `pipelines/prediction_pipeline.py` Defines `PredictPipeline` and `CustomData` helpers to transform a single row and predict using saved `preprocessor.pkl` and `model.pkl`.
 - `pipelines/training_pipeline.py` Placeholder file for orchestrating the training flow.
 - `artifacts/` Contains CSVs and serialized model artifacts produced by the pipeline (e.g., `train.csv`, `test.csv`, `model.pkl`, `preprocessor.pkl`).
 - `app.py` FastAPI backend application serving the model as a REST API with endpoints for health checks and predictions.
 - `templates/index.html` Frontend HTML form for interactive churn predictions.
 - `Dockerfile` Containerizes the FastAPI app and artifacts for deployment.
 - `README.md` Project overview and Hugging Face Spaces configuration (e.g., `sdk: docker`, `app_port: 7860`).
-

0.6 Data

- The main dataset used for experiments is the **Telco Customer Churn dataset**, located at: `notebook/data/WA_Fn-UseC_-Telco-Customer-Churn.csv` (Available on Kaggle: <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>)
- **Key columns:**
 - **Numeric:** `tenure`, `MonthlyCharges`, `TotalCharges`
 - **Categorical:** Various service, contract type, and payment method fields. The target variable is `Churn`.

Notes from preprocessing:

- `customerID` is dropped (non-predictive identifier).
 - `TotalCharges` contains blank entries coerced to numeric and imputed with the median.
 - `No internet service` and `No phone service` categories are replaced with `No`.
 - Binary `Yes/No` features are mapped to 1/0.
 - `gender` is mapped to 0/1, and `SeniorCitizen` is converted to integer.
-

0.7 Core Module Responsibilities (Detailed)

1. `src/Customer_Churn_Prediction/utils.py`
 - `read_csv_data()`: Reads the input CSV used by notebook flows.
 - `save_object(file_path, obj)` / `load_object(file_path)`: Helper functions for pickling and unpickling Python objects.
 - `evaluate_models(X_train, y_train, X_test, y_test, models, param)`: Runs `RandomizedSearchCV` or grid search for each model, returning scores and best parameters.
2. `src/Customer_Churn_Prediction/components/data_ingestion.py`
 - `DataIngestion.initiate_data_ingestion()`:
 - Reads raw CSV from `notebook/data`.
 - Writes raw data to `artifacts/raw.csv`.
 - Splits into training (80%) and test (20%) sets, saving them as `artifacts/train.csv` and `artifacts/test.csv`.
 - Returns file paths to the train and test CSVs.
3. `src/Customer_Churn_Prediction/components/data_transformation.py`
 - `TelcoDataCleaner`: Custom scikit-learn transformer that:
 - Drops the `customerID` column.

- Converts `TotalCharges` to numeric and fills missing values with the median.
- Replaces `No internet service` / `No phone service` with `No`.
- Maps `Yes/No` columns to `1/0` and `gender` to `0/1`.
- Label-encodes `InternetService`, `Contract`, and `PaymentMethod` using fitted encoders.
- `DataTransformation.get_data_transformer_object()`:
 - Builds a `ColumnTransformer` that scales numeric columns (`tenure`, `MonthlyCharges`, `TotalCharges`) and passes through other features.
- `DataTransformation.initiate_data_transformation(train_path, test_path)`:
 - Reads train and test CSVs.
 - Fits the `TelcoDataCleaner` on training features and transforms both datasets.
 - Encodes the target (`Churn`) as binary (`1/0`).
 - Applies SMOTE to balance the training set and scales features.
 - Saves the combined preprocessor (cleaner + scaler pipeline) as `artifacts/preprocessor.pkl`.
 - Returns `train_arr`, `test_arr`, and the preprocessor path.
- 4. `src/Customer_Churn_Prediction/components/model_trainer.py`
 - `ModelTrainer.initiate_model_trainer(train_array, test_array)`:
 - Splits arrays into `X` and `y` (last column as target).
 - Defines candidate models Logistic Regression, Random Forest, SVM, AdaBoost, XGBoost.
 - Uses `evaluate_models()` to tune and evaluate models via `RandomizedSearchCV`.
 - Logs metrics via MLflow/Dagshub integration.
 - Saves the best-performing model as `artifacts/model.pkl` using `save_object()`.
- 5. `src/Customer_Churn_Prediction/pipelines/prediction_pipeline.py`
 - `PredictPipeline.predict(features)`: Loads `artifacts/model.pkl` and `artifacts/preprocessor.pkl`, transforms input features, and returns predictions.
 - `CustomData`: Helper class that converts a single user input row into a pandas `DataFrame` matching the model's expected schema.

0.8 Step-by-step: Reproduce the Notebook EDA & Modeling (Recommended Quick Steps)

1. **Install dependencies.** Ensure Python and required packages are available. Use the provided `requirements.txt` file:

```
pip install -r requirements.txt
```

2. **Run the Jupyter notebook.** Open and execute `customer.ipynb` to reproduce the exploratory and modeling steps. The notebook covers:

- Loading and inspecting the dataset.
- Cleaning categorical and numerical features.
- Encoding target and feature labels.
- Applying SMOTE for class imbalance.
- Training baseline models and evaluating via cross-validation.
- Saving the preprocessor and trained model artifacts.

3. **Automate the full pipeline.** To reproduce results without opening the notebook, run the pipeline directly from the command line:

```
# Run data ingestion, transformation, and model training  
python -m src.Customer_Churn_Prediction.components.data_ingestion
```

This command sequentially triggers the ingestion script, which then calls the transformation and model training modules internally. After successful execution, the following artifacts will be generated:

- `artifacts/preprocessor.pkl` serialized preprocessing pipeline.
 - `artifacts/model.pkl` trained model ready for inference.
-

0.9 Deployment & Serving (Hugging Face Spaces, FastAPI, Docker)

The project is deployed as a Docker container on **Hugging Face Spaces**, serving predictions through a **FastAPI** backend. A live demo is available at: <https://snehangshu270219-customer-churn-prediction-02.hf.space/>, where users can enter customer data via an interactive web form and receive churn predictions instantly.

0.9.1 FastAPI Design

- **Endpoints:**
 - GET `/` Serves the interactive HTML form (`templates/index.html`) for churn predictions.
 - GET `/health` Returns API health status and model readiness.
 - POST `/predict` Accepts a JSON payload following the `ChurnPredictionInput` schema, passes it through `PredictPipeline`, and returns churn prediction with probability and message.
- At startup, the app loads:
 - `artifacts/preprocessor.pkl` Data preprocessing pipeline.
 - `artifacts/model.pkl` Trained model for inference.

- Input validation is handled using **Pydantic models**, and errors are processed through a custom exception handler.

0.9.2 Docker Setup

The `Dockerfile` builds a self-contained image bundling the FastAPI app, dependencies, and artifacts. A few configurations were modified for deployment on **Hugging Face Spaces**, following their official Docker guidelines available here: <https://huggingface.co/docs/hub/spaces-sdks-docker>

Key features:

- Runs as a non-root user (`uid 1000`) for Spaces compatibility.
- Copies `artifacts/`, `templates/`, and all source code with correct permissions.
- Installs dependencies from `requirements.txt` using pip caching for faster builds.
- Exposes port 7860 and runs Uvicorn on `0.0.0.0:7860`.

Build and Run Locally:

```
# Base image
FROM python:3.10-slim

# Create non-root user required by Spaces (uid 1000)
RUN useradd -m -u 1000 user
USER user
ENV HOME=/home/user PATH=/home/user/.local/bin:$PATH
WORKDIR $HOME/app

# Install dependencies with caching
COPY --chown=user requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copy project files (ensure templates/ and static/ are included)
COPY --chown=user . .

# Expose Spaces port
EXPOSE 7860

# Health-friendly environment defaults (optional)
ENV PYTHONUNBUFFERED=1 \
    UVICORN_WORKERS=1

# Start FastAPI (adjust module:variable if not app:app)
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "7860"]
```

0.9.3 Hugging Face Spaces Deployment

- **SDK Configuration:** The `README.md` includes YAML front matter specifying:
 - `sdk:` `docker`
 - `app_port:` `7860`

These settings tell Spaces to deploy the Docker image and route traffic properly.

- **Build Process:** Every push to the Space repository triggers an automatic Docker build. The container then runs the FastAPI app, serving both UI and API endpoints.
 - **Interactive Demo:** The root route (`/`) displays a user-friendly HTML form that posts JSON data to `/predict`. The frontend and backend communicate on the same origin, avoiding CORS issues.
 - **Monitoring:** Use Hugging Face Space logs for debugging. The `/health` endpoint provides a lightweight readiness check.
 - **CI/CD Integration:** GitHub Actions can be configured to automatically sync the latest version to the Space using an `HF_TOKEN` secret and a git push to the Space repository.
-

0.10 How I Built the Telco Churn Prediction Project A Step-by-Step Blog

This section walks through how I developed the **Telco Customer Churn Prediction System** from scratch covering exploratory analysis, feature engineering, model selection, and the final production-ready deployment pipeline.

It reflects my process while working in this repository: starting from raw data exploration in `customer.ipynb`, making key design and engineering decisions, running and evaluating multiple models, and finally packaging everything into a modular, reproducible codebase under `src/Customer_Churn_Prediction`.

If you're here for a quick overview, jump to the **TL;DR** summary at the end. If you'd like to follow along or reproduce the results, check the **"How to Reproduce"** section below it includes practical commands and notebook steps for replicating the entire workflow.

0.11 TL;DR What I Accomplished

- Cleaned and preprocessed the Telco Customer Churn dataset.
- Consolidated categorical labels and fixed numeric parsing issues.
- Applied `StandardScaler` and `SMOTE` to prepare training data without data leakage.
- Trained and tuned multiple models Logistic Regression, Random Forest, SVM, AdaBoost, XGBoost, Stacking, and a small ANN and evaluated them using ROC-AUC and other classification metrics.

- Packaged the entire pipeline (*ingestion* → *transformation* → *training* → *prediction*) into modular components and saved reusable artifacts (`artifacts/preprocessor.pkl`, `artifacts/model.pkl`).
 - Deployed the model as a Dockerized FastAPI application on Hugging Face Spaces for both web-based and API-driven predictions.
-

0.12 Motivation and Initial Questions

I wanted a repeatable and auditable process that starts from a simple CSV file and ends with a trained model capable of making reliable churn predictions. The Jupyter notebook served as my initial sandbox for exploration. Once the experiments produced stable and interpretable results, I transitioned everything into modular Python scripts so the full pipeline could be rerun, extended, or integrated into production systems. Finally, I wrapped the workflow with a FastAPI layer for real-world inference and deployment.

Key questions that guided my work:

- How should blanks and missing values in numeric fields (e.g., `TotalCharges`) be handled safely?
 - Which categorical encoding strategy ensures consistency between training and inference?
 - How can I address class imbalance without introducing data leakage?
 - Which algorithms provide the best balance between interpretability and performance?
 - What is the cleanest way to containerize and deploy the final solution for others to use easily?
-

0.13 Data Cleaning & Preprocessing What I Actually Did

This section explains the exact preprocessing and feature engineering steps I performed before model training. Each step ensured that the data remained consistent, informative, and free from leakage between training and test sets.

1. **Drop non-informative ID column** The `customerID` column was removed since it's an identifier and not predictive of churn.
2. **Fix `TotalCharges` and handle blanks** The `TotalCharges` column contained some empty strings. I converted it to a numeric type (coercing errors to NaN) and imputed missing values with the median.

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)
```

3. **Simplify redundant categories** Several service-related columns used redundant tokens like `No internet service` or `No phone service`. I replaced these with `No` to simplify categorical encoding.

```
df.replace({'No internet service': 'No', 'No phone service': 'No'}, inplace=True)
```


4. **Encode binary and Yes/No features** All Yes/No fields (including `Churn`) were mapped to 1/0. Similarly, `gender` was encoded as 0/1 for consistency.
5. **Label-encode remaining categorical features** I used `LabelEncoder` for categorical columns like `InternetService`, `Contract`, and `PaymentMethod`. The encoders were fit on the training data and reused during inference to ensure consistent mappings.
6. **Scale numeric features** I applied `StandardScaler` to normalize `tenure`, `MonthlyCharges`, and `TotalCharges`. This helped improve model convergence and comparability between features.
7. **Handle class imbalance safely with SMOTE** I applied `SMOTE` (Synthetic Minority Over-sampling Technique) only on the training data *after scaling*. This helped balance classes while keeping the test set untouched for fair evaluation.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled, y_train)
```

Note: SMOTE must only be applied on the training set applying it before splitting causes synthetic samples to leak into the test set, invalidating evaluation metrics.

0.14 Modeling Experiments - What I Tried and Why

After completing data preprocessing, I experimented with several models to find the right balance between predictive performance, interpretability, and robustness. The goal was to compare traditional algorithms, ensemble methods, and a small neural network baseline under a consistent evaluation framework.

Models explored:

- **Logistic Regression** - served as the baseline model. Fast to train, interpretable, and easy to explain to non-technical audiences.
- **Random Forest** - strong all-rounder that handles non-linearities and mixed data types effectively.
- **XGBoost** - gradient boosting algorithm known for excellent performance on tabular data.
- **AdaBoost** - ensemble of weak learners designed to improve recall and precision.
- **SVM (Support Vector Machine)** - non-linear classifier providing good margins, though slower with large datasets.
- **Stacking Classifier** - combined multiple strong learners (e.g., Random Forest, XGBoost, Logistic Regression) for improved performance.

Hyperparameter Tuning: For model optimization, I used `RandomizedSearchCV` wrapped with `StratifiedKFold` cross-validation. This ensured that each fold maintained the same churn/non-churn class ratio, reducing bias from class imbalance.

Evaluation Metrics: Each model was evaluated using ROC-AUC, accuracy, precision, recall, and F1-score. I also analyzed confusion matrices and experimented with varying classification thresholds on predicted probabilities to adjust for business objectives (e.g., prioritizing recall to minimize customer retention loss).

Code excerpt (model evaluation and selection):

```
# utils.evaluate_models runs GridSearchCV or RandomizedSearchCV for each model
models = {
    'Logistic Regression': LogisticRegression(...),
    'Random Forest': RandomForestClassifier(...),
    'XGBoost': XGBClassifier(...),
    'SVM': SVC(...),
}

params = {
    'Logistic Regression': { ... },
    'Random Forest': { ... },
    'XGBoost': { ... },
    'SVM': { ... }
}

report, best_params = evaluate_models(X_train, y_train, X_test, y_test, models, params)
```

The training pipeline identified **Random Forest** as the best-performing model after evaluating all candidates. It consistently achieved the highest ROC-AUC and F1-score, showing a strong balance between recall and precision. All model performance logs and comparison metrics can be viewed in the MLflow tracking dashboard: [View MLflow Experiment on DagsHub](#)

The final model was serialized and saved for deployment using:

```
save_object('artifacts/model.pkl', best_model)
```

0.15 Engineering the Pipeline Files and Flow

After validating the end-to-end process in the notebook, I converted the workflow into modular, reusable components. Each module is responsible for a single, well-defined task making the entire system easier to maintain, debug, and extend.

Core components:

- `src/Customer_Churn_Prediction/components/data_ingestion.py` Reads the raw dataset and creates three files: `artifacts/train.csv`, `artifacts/test.csv`, and `artifacts/raw.csv`.
- `src/Customer_Churn_Prediction/components/data_transformation.py` Contains the `TelcoDataCleaner` class that applies cleaning rules and categorical transformations. It is integrated into a numeric pipeline (imputation + scaling). The combined preprocessor is serialized as `artifacts/preprocessor.pkl`.

- `src/Customer_Churn_Prediction/components/model_trainer.py` Orchestrates model training and hyperparameter tuning using MLflow (with Dagshub integration). The best-performing model is saved as `artifacts/model.pkl`.
- `src/Customer_Churn_Prediction/pipelines/prediction_pipeline.py` Defines `PredictPipeline`, which loads the preprocessor and trained model to make predictions on new data. The `CustomData` helper class converts user input into a `DataFrame` suitable for inference.

Why this layout? The structure follows a clear separation of concerns:

- **Ingestion** Handles file I/O and dataset splits.
- **Transformation** Performs all feature engineering and ensures the same transformations are reused at inference.
- **Model Training** Focuses on selection, tuning, evaluation, and persistence.
- **Prediction Pipeline** Provides a lightweight inference wrapper for the final deployed model.

This modular design allows reproducibility and smooth integration with both notebook-based experimentation and automated CI/CD pipelines.

0.16 How to Reproduce Everything Locally (Copy-Paste Commands)

All steps below can be executed directly from the repository root in **Windows PowerShell** (or terminal on Linux/Mac).

1) Install dependencies

```
pip install -r requirements.txt
```

2) Run the full pipeline (ingestion → transformation → training)

```
python -m src.Customer_Churn_Prediction.components.data_ingestion
```

This single command will:

- Create `artifacts/train.csv` and `artifacts/test.csv`
- Save the preprocessing pipeline as `artifacts/preprocessor.pkl`
- Train and serialize the model as `artifacts/model.pkl`

3) Test the FastAPI app locally (after pipeline run)

```
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

Then visit:

- `http://localhost:8000` interactive web form
- `http://localhost:8000/docs` API documentation (Swagger UI)

4) Run with Docker (alternative deployment)

```
docker build -t snehangshu2002/churn-prediction-system:latest . && \
docker run -d -p 8000:8000 snehangshu2002/churn-prediction-system:latest
```

The container will start the FastAPI app at <http://localhost:7860> and serve both the HTML interface and API routes.

0.17 Practical Lessons and Decisions I Defend

Throughout this project, I made several engineering choices that balanced reliability, clarity, and reproducibility. These are the lessons I found worth defending in future projects.

- Keep all data cleaning logic inside a custom transformer class so that both training and inference use identical preprocessing steps.
 - Apply *SMOTE* *only* to the training set and after feature scaling to avoid data leakage.
 - Always save both the preprocessor and the trained model as serialized artifacts so they can be reloaded reliably in production.
 - Use cross-validated **RandomizedSearchCV** instead of exhaustive grid search it's faster and generally finds near-optimal hyperparameters.
 - Deploy with Docker and use relative paths for frontend backend communication to avoid CORS issues on platforms like Hugging Face Spaces.
-

0.18 Results & Variability

During model experimentation, tree-based algorithms particularly **Random Forest**, **XGBoost**, and **Stacking Ensembles** consistently outperformed simpler models. They provided the best balance between ROC-AUC, precision, and recall.

Performance metrics varied slightly across runs due to random initialization and SMOTE sampling randomness. To reproduce exact results, fix all random seeds and persist the final fitted model object.

The deployed Hugging Face Space delivers real-time predictions with low latency, making it suitable for interactive demos and lightweight API integrations.
